

# Shell Scripting Practice MCQs

---

## Part 1: Questions

1. Q1. Which command correctly checks if a file is both readable and not empty?

A) `[ -r file.txt ]`

B) `[ -s file.txt ]`

C) `[ -r file.txt -a -s file.txt ]` ✓

D) `[ -s file.txt -o -r file.txt ]`

2. Q2. What will this script print?

```
var=hello
```

```
if [ "$var" = hello ]; then echo yes; fi
```

A) yes ✓

B) hello

C) nothing

D) error

3. Q3. Which of the following is a valid way to define a function in Bash?

A) `def myfunc() {}`

B) `function myfunc {}` ✓

C) `myfunc = function {}`

D) `define myfunc {}`

4. Q4. What will `[[ "abc" > "bcd" ]]` return in Bash?

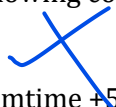
A) True

B) False ✓

C) Error

D) abc

5. Q5. Which of the following commands deletes `.tmp` files older than 5 days?

- A) `rm *.tmp -mtime +5` 
- B) `find . -name "*.tmp" -mtime +5 -delete` 
- C) `delete *.tmp +5days`
- D) `rm -rf *.tmp`

6. Q6. Which Bash loop syntax is correct for iterating 5 times?

- A) `for i in 1 2 3 4 5; do echo $i; done` 
- B) `for (i=1;i<=5;i++)`
- C) `loop(1,5)`
- D) `repeat 5 times`

7. Q7. What is the purpose of ``trap 'echo Bye' EXIT``?

- A) Ignore exit
- B) Handle Ctrl+C
- C) Execute 'echo Bye' before exit 
- D) Trap background jobs

8. Q8. What will happen?

```
name="John Doe"  
echo Hello $name
```

- A) Error: space in variable 
- B) Hello John Doe 
- C) Hello \$name
- D) Hello "John"

9. Q9. Which is the correct syntax for reading user input into a variable?

- A) `input var`
- B) `scanf var`
- C) `read var` 
- D) `echo var`

10. Q10. What is wrong with this line?

```
if [ $value == 10 ]; then echo Match; fi
```

- A) `==` is invalid 
- B) Quotes are missing 
- C) `then` should be `do`
- D) Needs elif

11. Q11. Which command lists files by size in human-readable format?

- A) `ls -lhS` 
- B) `du -a`
- C) `ls -ltr`
- D) `find -type f`

12. Q12. How can you check if a directory exists?

- A) `[ -f mydir ]`
- B) `test -d mydir` 
- C) `[ exists mydir ]`
- D) `[ -e mydir ]` 

13. Q13. What does `chmod +x script.sh` do?

- A) Deletes the script
- B) Renames script
- C) Makes it executable 
- D) Compresses it

14. Q14. How to store the output of a command in a variable?

- A) `output = `ls``
- B) `var=$(ls)` 
- C) `echo ls > var`
- D) `read var < ls`

15. Q15. Which loop construct allows use of index and element?

- A) for i; do
- B) for i in seq
- C) for i in \${arr[@]}
- D) for i in \$(seq 1 5); do ✓

16. Q16. Which of these conditions checks if a string is non-empty?

- A) [ -e \$str ]
- B) [ -n "\$str" ] ✓
- C) [ \$str != null ]
- D) [ notempty \$str ]

17. Q17. What will happen here?

```
rm -rf /tmp/*
```

- A) Deletes /tmp
- B) Deletes contents of /tmp ✓
- C) Deletes root
- D) Nothing

18. Q18. How to make a script fail if any command fails?

- A) set -e ✓
- B) failonerror=1
- C) trap exit
- D) set safe

19. Q19. What will this output?

```
for f in *.log; do echo $f; done
```

- A) All files
- B) Only .log files ✓
- C) Error if none
- D) The literal '\*.log'

20. Q20. Which command is used to monitor disk usage of a directory?

- A) df
- B) du ✓
- C) ls -lh
- D) top

21. Q21. What does this line mean?

`echo "${date}"`

- A) Prints the string `date`
- B) Prints nothing
- C) Prints today's date ✓
- D) Shows error

22. Q22. How can you redirect both stdout and stderr to a file?

- A) 2> file
- B) > file
- C) &> file ✓
- D) >& file

23. Q23. What is the effect of `export VAR=value`?

- A) Creates variable
- B) Makes it available to child processes ✓
- C) Deletes VAR
- D) Sets value permanently

24. Q24. Which operator checks for equality in double brackets?

- A) == ✓
- B) eq
- C) :=
- D) !=

25. Q25. How do you pass arguments to a script?

A) read var

B) ./script.sh value1 value2 ✓

C) pipe values

D) using scanf

26. Q26. What will `echo \$0` output inside a script?

A) First argument ✓✗

B) Filename of the script ✓

C) Current user

D) Last command

27. Q27. What does `basename /usr/local/bin/script.sh` output?

A) /usr/local/bin

B) bin

C) script.sh ✓

D) .sh

28. Q28. What is the bug in this script?

```
count=0
while [ $count -lt 3 ]; do
    echo $count
    count=$((count + 1))
done
```

A) Variable should not use \$ in assignment

B) `-lt` should be `<=`

C) Missing 'do'

D) No bug, this runs fine ✓

29. Q29. What is the issue with this line?

```
if [ $var == "test" ]; then echo Match; fi
```

- A) `==` is not supported
- B) \$var should be quoted ✓
- C) Should use `-eq`
- D) `then` is missing

30. Q30. What will happen when this is executed?

```
filename=myfile.txt
if [ -e $filename ]; then echo Exists; fi
```

- A) Always prints Exists
- B) Syntax error
- C) May error if file doesn't exist ✓
- D) Errors if filename has spaces ✓

31. Q31. Why does this fail?

```
value=5
if [ $value -gt 3 ]
then
  echo High
fi
```

- A) `-gt` invalid
- B) Missing quotes around `\$value`
- C) Syntax is wrong
- D) It doesn't fail ✓

32. Q32. Identify the bug:

```
for i in {1..5}
  echo $i
done
```

- A) Missing `do` ✓
- B) Incorrect brace expansion ✓
- C) \$i not initialized
- D) `done` misplaced

## Part 2: Answer Key & Explanations

Q1. Answer: C

Explanation: ``-r`` checks if readable, ``-s`` if size > 0. ``-a`` combines both correctly.

Q2. Answer: A

Explanation: Correctly quoted variable; condition matches the string 'hello'.

Q3. Answer: B

Explanation: ``function myfunc {}`` is a valid bash function declaration.

Q4. Answer: B

Explanation: ``>`` does lexicographical comparison in ``[[ ]]`; 'abc' is less than 'bcd'.

Q5. Answer: B

Explanation: Only ``find`` with ``-mtime`` and ``-delete`` targets files by age.

Q6. Answer: A

Explanation: This is the correct and idiomatic Bash loop syntax.

Q7. Answer: C

Explanation: The EXIT signal is triggered when the script ends.

Q8. Answer: B

Explanation: Variable expands with space unless quoted—here it's fine.



Q9. Answer: C

Explanation: ``read`` is the correct command to accept input from the user.

Q10. Answer: B

Explanation: If ``$value`` is empty, this results in a syntax error.

Q11. Answer: A

Explanation: ``-l`` long listing, ``-h`` human-readable, ``-S`` sort by size.

Q12. Answer: B

Explanation: ``-d`` checks for directories.

Q13. Answer: C

Explanation: ``+x`` adds the execute permission.

Q14. Answer: B

Explanation: ``$(...)`` captures the output of the command.

Q15. Answer: D

Explanation: ``seq`` generates numbers; useful for indexed iteration.

Q16. Answer: B

Explanation: ``-n`` checks for non-zero length strings.

Q17. Answer: B

Explanation: The wildcard expands to all contents, not the directory itself.

Q18. Answer: A

Explanation: ``set -e`` causes immediate exit on failure.

Q19. Answer: B

Explanation: Globbing expands to matching files.

Q20. Answer: B

Explanation: ``du`` reports disk usage.

Q21. Answer: C

Explanation: Command substitution prints output of ``date``.

Q22. Answer: C

Explanation: ``&>`` combines both streams in Bash.

Q23. Answer: B

Explanation: ``export`` makes the variable accessible to sub-shells.

Q24. Answer: A

Explanation: ``==`` is supported inside ``[[ ]]`.

Q25. Answer: B

Explanation: Arguments are accessed using ``$1``, ``$2``, etc.

Q26. Answer: B

Explanation: ``$0`` refers to the name of the script being executed.

Q27. Answer: C

Explanation: ``basename`` strips the path, returns the file name.

Q28. Answer: D

Explanation: This is valid Bash syntax. The variable assignment and loop are correct.

Q29. Answer: B

Explanation: If ``$var`` is empty, the condition breaks. Always quote variables in ``[ ]``.

Q30. Answer: D

Explanation: If ``filename`` contains spaces and is unquoted, test fails or misbehaves.

Q31. Answer: D

Explanation: This is a valid numeric comparison; no issue if ``$value`` is set.

Q32. Answer: A

Explanation: ``do`` is required after the ``for`` loop declaration. Missing here.